



Universidade de Brasília (UNB)
Departamento de Ciência da Computação (CIC)
CIC 0105 - ENGENHARIA DE SOFTWARE - Turma 01 - 2026/1
Professor: Fernando Antonio de Araujo Chacon de Albuquerque

Alunos - Matrículas:
Caetano Korilo Fleury de Amorim - 212006737
Luis Henrique Wiltgen de Toledo - 221003968
Pedro Rafael Faria Ferreira - 190094591
Vinicius Chaves - 211060764

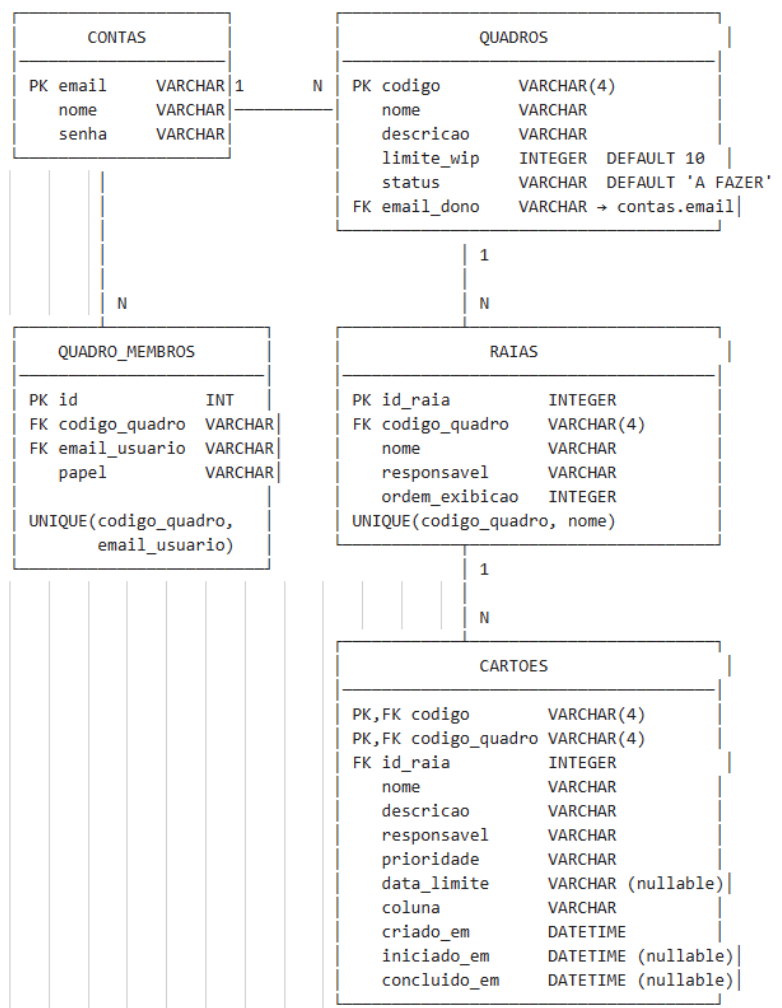
Projeto Físico de Banco de Dados

Kanban Web

1. Visão Geral

O banco de dados do Kanban Web é composto por 5 tabelas que armazenam usuários, projetos, membros, raias e tarefas. O arquivo físico é kanban_web/kanban.db, criado automaticamente na primeira execução da aplicação.

2. Diagrama Entidade-Relacionamento (DER)



3. Especificação das Tabelas

3.1 Tabela contas



Propósito: Armazenar as credenciais e dados de identificação dos usuários.

Coluna	Tipo	Restrição	Descrição
email	VARCHAR	PRIMARY KEY, INDEX	E-mail do usuário; identificador único global
nome	VARCHAR	NOT NULL	Nome completo do usuário
senha	VARCHAR	NOT NULL	Senha em texto plano (restrição acadêmica)

DDL:

```
SQL
CREATE TABLE contas (
    email VARCHAR NOT NULL PRIMARY KEY,
    nome VARCHAR NOT NULL,
    senha VARCHAR NOT NULL
);
CREATE INDEX ix_contas_email ON contas (email);
```

Cardinalidade: Uma conta pode possuir N quadros (como dona) e participar de N quadros (como membro).

Exemplo de Registro:

```
email = "pedro@unb.br"
nome = "Pedro Alves"
senha = "Ab1.x"
```

3.2 Tabela quadros

Propósito: Armazenar os projetos Kanban com seus metadados e configurações.

Coluna	Tipo	Restrição	Descrição
codigo	VARCHAR(4)	PRIMARY KEY, INDEX	Código único do quadro no formato LLDD
nome	VARCHAR	NOT NULL	Nome descritivo do projeto
descricao	VARCHAR	NOT NULL	Descrição resumida do objetivo do projeto
limite_wip	INTEGER	NOT NULL, DEFAULT 10	Limite WIP por coluna;



Coluna	Tipo	Restrição	Descrição
			valores: 5, 10, 15, 20
status	VARCHAR	NOT NULL, DEFAULT 'A FAZER'	Status calculado do projeto; ver domínio abaixo
email_dono	VARCHAR	NOT NULL, FK → contas.email	E-mail do dono do quadro

Domínio `status`:

- 'A FAZER' - todos os cartões em A FAZER
- 'EM ANDAMENTO' - cartões em colunas distintas
- 'EM TESTES' - todos em EM TESTE ou FEITO
- 'CONCLUIDO' - todos em FEITO

DDL:

```
SQL
CREATE TABLE quadros (
  codigo  VARCHAR(4) NOT NULL PRIMARY KEY,
  nome    VARCHAR  NOT NULL,
  descricao VARCHAR NOT NULL,
  limite_wip INTEGER NOT NULL DEFAULT 10,
  status  VARCHAR  NOT NULL DEFAULT 'A FAZER',
  email_dono VARCHAR NOT NULL,
  FOREIGN KEY (email_dono) REFERENCES contas(email)
);
CREATE INDEX ix_quadros_codigo ON quadros (codigo);
```

Cascade: Ao excluir uma conta, todos os seus quadros são excluídos em cascata (cascade="all, delete-orphan" no ORM).

Exemplo de Registro:

```
codigo   = "AB12"
nome     = "Projeto Alpha"
descricao = "Sistema web de Kanban"
limite_wip = 10
```



```
status = "EM ANDAMENTO"
email_dono = "pedro@unb.br"
```

3.3 Tabela quadro_membros

Propósito: Tabela associativa que registra quais usuários participam de quais quadros, com seu respectivo papel.

Coluna	Tipo	Restrição	Descrição
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Surrogate key da associação
codigo_quadro	VARCHAR(4)	NOT NULL, FK → quadros.codigo	Quadro ao qual o membro pertence
email_usuario	VARCHAR	NOT NULL, FK → contas.email	E-mail do usuário membro
papel	VARCHAR	NOT NULL, DEFAULT 'MEMBRO'	Papel do usuário no quadro

Domínio `papel`:

- 'DONO' - criador do quadro (registrado na tabela de membros apenas para listagem; a autoridade real vem de quadros.email_dono)
- 'MEMBRO' - usuário convidado com acesso ao quadro

Constraint de unicidade: Um usuário não pode ser membro duplicado do mesmo quadro.

DDL:

```
SQL
CREATE TABLE quadro_membros (
    id            INTEGER    NOT NULL PRIMARY KEY
    AUTOINCREMENT,
    codigo_quadro VARCHAR(4) NOT NULL,
    email_usuario VARCHAR    NOT NULL,
    papel         VARCHAR    NOT NULL DEFAULT 'MEMBRO',
    FOREIGN KEY (codigo_quadro) REFERENCES quadros(codigo),
    FOREIGN KEY (email_usuario) REFERENCES contas(email),
```



```
CONSTRAINT uq_quadro_membro UNIQUE (codigo_quadro,  
email_usuario)  
);
```

Observação: O dono do quadro não é inserido nesta tabela automaticamente - a lógica de listagem de membros em routers/membros.py adiciona o dono à lista de resposta dinamicamente. Apenas membros convidados ficam nesta tabela.

Exemplo de Registro:

```
id          = 1  
codigo_quadro = "AB12"  
email_usuario = "ana@unb.br"  
papel       = "MEMBRO"
```

3.4 Tabela raia

Propósito: Organizar horizontalmente o quadro por responsável (swimlanes).

Coluna	Tipo	Restrição	Descrição
id_raia	INTEGER	PRIMARY KEY, AUTOINCREMENT	Identificador único da raia
codigo_quadro	VARCHAR(4)	NOT NULL, FK → quadros.codigo	Quadro ao qual a raia pertence
nome	VARCHAR	NOT NULL	Nome da raia
responsavel	VARCHAR	NOT NULL	Responsável associado à raia
ordem_exibicao	INTEGER	NOT NULL, DEFAULT 0	Ordem visual da raia no quadro

Constraint de unicidade: Não pode existir nome de raia duplicado no mesmo quadro.

DDL:

```
SQL  
CREATE TABLE raia (  
    id_raia          INTEGER    NOT NULL PRIMARY KEY  
    AUTOINCREMENT,  
    codigo_quadro    VARCHAR(4) NOT NULL,
```



```
nome          VARCHAR NOT NULL,
responsavel   VARCHAR NOT NULL,
ordem_exibicao INTEGER NOT NULL DEFAULT 0,
FOREIGN KEY (codigo_quadro) REFERENCES quadros(codigo),
              CONSTRAINT uq_raia_nome_no_quadro UNIQUE
(codigo_quadro, nome)
);
CREATE INDEX ix_raias_codigo_quadro ON raias (codigo_quadro);
```

Exemplo de Registro:

```
id_raia      = 3
codigo_quadro = "AB12"
nome         = "Pedro Alves"
responsavel  = "Pedro Alves"
ordem_exibicao = 1
```

3.5 Tabela cartoes

Propósito: Armazenar as tarefas (cartões) associadas a um quadro, com seu estado no fluxo Kanban e carimbos de tempo de ciclo.

Coluna	Tipo	Restrição	Descrição
codigo	VARCHAR(4)	PK (composta com codigo_quadro), INDEX	Código do cartão no formato LLDD
codigo_quadro	VARCHAR(4)	PK (composta), FK → quadros.codigo	Quadro ao qual o cartão pertence
id_raia	INTEGER	NOT NULL, FK → raias.id_raia	Raia horizontal à qual o cartão pertence
nome	VARCHAR	NOT NULL	Título da tarefa
descricao	VARCHAR	NOT NULL	Detalhamento da tarefa
responsavel	VARCHAR	NOT NULL	Nome do membro responsável pela tarefa
prioridade	VARCHAR	NOT NULL, DEFAULT 'MEDIA'	Nível de prioridade; domínio: BAIXA, MEDIA, ALTA



Coluna	Tipo	Restrição	Descrição
data_limite	VARCHAR	NULL	Prazo no formato ISO 8601 (YYYY-MM-DD); opcional
coluna	VARCHAR	NOT NULL, DEFAULT 'A FAZER'	Estágio atual no fluxo Kanban
criado_em	DATETIME	DEFAULT utcnnow()	Timestamp UTC de criação do cartão
iniciado_em	DATETIME	NULL	Timestamp UTC da primeira entrada em FAZENDO
concluido_em	DATETIME	NULL	Timestamp UTC da primeira entrada em FEITO

Domínio `prioridade`: 'BAIXA', 'MEDIA', 'ALTA'

Domínio `coluna`: 'A FAZER', 'FAZENDO', 'EM TESTE', 'FEITO'

Chave Primária Composta: O par (codigo, codigo_quadro) é a PK, permitindo que dois quadros diferentes tenham cartões com o mesmo código.

DDL:

```
SQL
CREATE TABLE cartoes (
  codigo      VARCHAR(4) NOT NULL,
  codigo_quadro VARCHAR(4) NOT NULL,
  id_raia     INTEGER  NOT NULL,
  nome        VARCHAR  NOT NULL,
  descricao   VARCHAR  NOT NULL,
  responsavel VARCHAR  NOT NULL,
  prioridade  VARCHAR  NOT NULL DEFAULT 'MEDIA',
  data_limite VARCHAR,
  coluna      VARCHAR  NOT NULL DEFAULT 'A FAZER',
  criado_em   DATETIME DEFAULT (datetime('now')),
  iniciado_em DATETIME,
```



```
concluido_em DATETIME,  
PRIMARY KEY (codigo, codigo_quadro),  
FOREIGN KEY (codigo_quadro) REFERENCES quadros(codigo),  
FOREIGN KEY (id_raia) REFERENCES raias(id_raia)  
);  
CREATE INDEX ix_cartoes_codigo ON cartoes (codigo);
```

Cascade: Ao excluir um quadro, todos os seus cartões são excluídos em cascata.

Exemplo de Registro:

```
codigo      = "CD34"  
codigo_quadro = "AB12"  
id_raia     = 3  
nome        = "Implementar API"  
descricao   = "Endpoints REST da aplicação"  
responsavel = "Pedro Alves"  
prioridade  = "ALTA"  
data_limite = "2026-05-20"  
coluna      = "FAZENDO"  
criado_em   = "2026-05-01 10:00:00"  
iniciado_em = "2026-05-03 14:30:00"  
concluido_em = NULL
```

4. Diagrama de Relacionamentos (Notação Crow's Foot)

```
CONTAS ||—o< QUADROS      (uma conta possui zero ou muitos quadros)  
CONTAS ||—o< QUADRO_MEMBROS (uma conta pode ser membro de zero ou muitos quadros)  
QUADROS ||—o< QUADRO_MEMBROS (um quadro pode ter zero ou muitos membros)  
QUADROS ||—o< RAIAS      (um quadro possui zero ou muitas raias)  
QUADROS ||—o< CARTOES     (um quadro possui zero ou muitos cartões)  
RAIAS ||—o< CARTOES      (uma raia possui zero ou muitos cartões)
```

5. Índices

Tabela	Índice	Tipo	Justificativa
contas	ix_contas_email	B-Tree	Busca de conta por e-mail (login, FK lookup)
quadros	ix_quadros_codigo	B-Tree	Busca de quadro por código (FK lookup, GET)
raias	ix_raias_codigo_quadro	B-Tree	Listagem de raias por quadro



Tabela	Índice	Tipo	Justificativa
cartoes	ix_cartoes_codigo	B-Tree	Busca de cartão por código
quadro_membros	uq_quadro_membro (UNIQUE)	B-Tree	Impede duplicidade; implícito em UNIQUE constraint

6. Constraints de Integridade

Tabela	Constraint	Tipo	Descrição
contas	PK email	PK	E-mail único e não nulo
quadros	PK codigo	PK	Código único e não nulo
quadros	FK email_dono → contas.email	FK	Integridade referencial; cascata de delete
quadro_membros	PK id	PK	Surrogate key
quadro_membros	FK codigo_quadro → quadros.codigo	FK	Integridade referencial; cascata de delete
quadro_membros	FK email_usuario → contas.email	FK	Integridade referencial; cascata de delete
quadro_membros	UNIQUE (codigo_quadro, email_usuario)	UNIQUE	Impede membro duplicado no mesmo quadro
raias	PK id_raia	PK	Identificador único da raia
raias	FK codigo_quadro → quadros.codigo	FK	Integridade referencial; cascata de delete
raias	UNIQUE (codigo_quadro, nome)	UNIQUE	Impede nome de raia duplicado no mesmo quadro
cartoes	PK (codigo, codigo_quadro)	PK (composta)	Código único por quadro
cartoes	FK codigo_quadro → quadros.codigo	FK	Integridade referencial; cascata de delete



Tabela	Constraint	Tipo	Descrição
cartoes	FK id_raia → raias.id_raia	FK	Garante pertencimento do cartão a uma raia válida

7. Estratégia de Migração

A criação das tabelas é feita automaticamente via:

```
PYTHON  
models.Base.metadata.create_all(bind=engine)
```

Para migrações incrementais (adição de colunas, alteração de tipos) em versões futuras, recomenda-se a adoção de Alembic:

```
BASH  
pip install alembic  
alembic init migrations  
alembic revision --autogenerate -m "adicionar campo labels em cartoes"  
alembic upgrade head
```

8. Considerações de Performance

Cenário	Estratégia
Listagem de cartões por quadro	Índice em codigo + filtro por codigo_quadro (PK)
Listagem de cartões por raia	Índice de raias.codigo_quadro + FK cartoes.id_raia
Listagem de quadros do usuário	Query de união (como dono) + subquery (como membro)
Verificação WIP ao mover cartão	COUNT(*) filtrado por codigo_quadro e coluna
Recalcular status do quadro	SELECT coluna FROM cartoes WHERE codigo_quadro = ?

9. Backup e Recuperação

Estratégia	Procedimento
Backup manual	Cópia do arquivo kanban.db para local seguro
Backup automático	Script cron: cp kanban.db kanban_backup_\$(date +%F).db
Recuperação	Substituir kanban.db pela cópia de backup e reiniciar



Estratégia	Procedimento
WAL Mode (recomendado)	PRAGMA journal_mode=WAL; para maior concorrência